Platform Architecture Overview

> **Document Owner:** Platform Architecture Team | **Status:** Approved | **Last Updated:** February 2026

Overview

This document defines the core architectural decisions, technology stack, and integration patterns for our enterprise data platform built on Azure Databricks. It serves as the navigation hub for all platform architecture rules, with detailed rules defined in their respective documents.

Golden Rules

This document owns the orphan platform rules (PA-01..05) and cost optimization rules (COST-01..03). All other platform rules are owned by their respective detail documents — see the Platform Rules Cross-Reference below.

ID	Rule	Severity	WAF Pillar
PA-01	No production data on DBFS root	Critical	Security
PA-02	Use system tables for platform observability	Required	OpEx
PA-03	Workspace isolation by environment and security boundary	Required	Security
PA-04	Use Lakehouse Federation for external data access	Recommended	Interop
PA-05	Use certified partner integrations	Recommended	Interop
COST-01	Budget policies for all production workspaces	Critical	Cost
COST-02	Quarterly cost optimization reviews via system tables	Required	Cost
COST-03	Balance always-on vs triggered streaming workloads	Recommended	Cost

---

Platform Rules Cross-Reference

All platform architecture rules organized by owner document:

Prefix	Document	Rules	Count
DL	UC Tables & Delta Lake	DL-01..12	12
SC	Serverless Compute	SC-01..09	9
CP	Classic Compute Governance	CP-01..09	9
SM	Secrets & Workspace	SM-01..04	4

Prefix	Document	Rules	Count
GOV	UC Governance	GOV-01..16	16
REL	Reliability & DR	REL-01..09	9
SEC	Network & Security	SEC-01..09	9
IN	Asset Bundle Standards	IN-01..13	13
PY	Python Development	PY-01..04	4
PA	This document	PA-01..05	5
COST	This document	COST-01..03	3

Architecture Principles

1. Lakehouse with Medallion Architecture

Data flows through three layers of increasing quality:

Layer	Purpose	Technology
Bronze	Raw data with CDF	Delta Lake + Auto Loader
Silver	Validated data	Delta Live Tables + Expectations
Gold	Business entities	YAML-driven MERGE

2. Unity Catalog Foundation

All assets managed through Unity Catalog's three-level namespace:

```
<catalog>.<schema>.<object>
```

Asset Type	Example
Tables	`company_prod.gold.dim_customer`
Views	`company_prod.semantic.cost_metrics`
Functions	`company_prod.semantic.get_daily_sales`
ML Models	`company_prod.ml.customer_churn_model`
Volumes	`company_prod.raw.landing_files`

3. Delta Lake with Automatic Clustering

Every table uses Delta Lake with automatic liquid clustering:

```sql
CREATE TABLE catalog.schema.my_table (...)
USING DELTA
CLUSTER BY AUTO  -- Let Databricks choose optimal clustering
TBLPROPERTIES (
    'delta.enableChangeDataFeed' = 'true',
    'delta.autoOptimize.optimizeWrite' = 'true',
    'delta.autoOptimize.autoCompact' = 'true'
);
```

4. Serverless-First Compute

All workloads default to serverless:

Workload	Compute	Benefit
SQL Analytics	Serverless SQL Warehouse	Instant, built-in Photon
ETL Jobs	Serverless Workflows	Auto-scaling, no config
DLT Pipelines	Serverless DLT	Efficient workflows
Interactive	Serverless Notebooks	Fast startup

Technology Stack

Core Platform

Component	Technology
Compute	Databricks Serverless
Storage	Delta Lake on Cloud
Governance	Unity Catalog
Orchestration	Databricks Workflows
CI/CD	Asset Bundles

Data Access

Component	Technology
Federation	Lakehouse Federation
Sharing	Delta Sharing

Semantic Layer

Component	Technology
Metrics	Metric Views (YAML)

Component	Technology
Functions	TVFs (SQL)
NL Interface	Genie Spaces
Dashboards	AI/BI Lakeview

ML/AI

Component	Technology
Experiments	MLflow
Feature Store	Unity Catalog
Model Registry	Unity Catalog
Model Serving	Serverless Endpoints
Agents	LangGraph + MLflow

Data Ingestion Strategy

> **See also:** Data Access Patterns (DI-01..07) — decision framework for Ingest vs Federate vs Share.

Priority order for ingestion approaches:

Priority	Approach	Use For
1	**Lakeflow Connect**	Salesforce, ServiceNow, SQL Server, Workday
2	**DLT + Auto Loader**	File-based sources, message buses
3	**Structured Streaming**	Complex real-time requirements only

Avoid: Custom batch pipelines with full loads (legacy only).

Catalog Structure

```
company_prod/
├── bronze/           # Raw data layer
├── silver/           # Validated layer
├── gold/             # Business entities
├── semantic/         # Metric Views, TVFs
├── ml/               # Features, models
├── monitoring/       # Monitoring outputs
└── sandbox/          # Ad-hoc exploration
```

Deployment Model

All infrastructure deployed via Asset Bundles:

```
Git Push → CI Build → Validate Bundle → Dev Deploy → Prod Deploy
```

Resources managed:

- Jobs
- Pipelines
- Schemas
- Functions
- Dashboards

PA-01: No DBFS Root for Production Data

Rule

Never store production data, checkpoints, or application artifacts on the DBFS root (`dbfs:/`). All production data must reside in Unity Catalog managed tables or external locations governed by UC.

Why It Matters

- DBFS root is shared across the workspace with no fine-grained access control
- Not governed by Unity Catalog (no lineage, no auditing, no tags)
- Data on DBFS root is tied to the workspace lifecycle
- Incompatible with workspace migration or multi-workspace architectures

Acceptable Locations

Use Case	Location
Production tables	Unity Catalog managed tables
Unstructured files	Unity Catalog Volumes
Temporary dev files	DBFS root (dev workspace only)
Streaming checkpoints	Cloud storage (ZRS/GRS) via UC external location

---

PA-02: System Tables for Observability

Rule

Use Databricks system tables (`system.*`) as the primary source for platform observability, cost tracking, audit, and operational monitoring.

Why It Matters

- Provides a unified view of compute, billing, queries, and audit events

- No additional infrastructure required
- Enables proactive cost management and anomaly detection
- Supports compliance and security monitoring

Key System Tables

Table	Purpose
`system.billing.usage`	Cost tracking by SKU, tags, workspace
`system.compute.clusters`	Cluster inventory and configuration
`system.access.audit`	Security audit events
`system.query.history`	SQL query performance and patterns
`system.lakeflow.pipeline_events`	DLT/SDP pipeline execution history
`system.serving.endpoint_usage`	Model serving token usage
`system.storage.predictive_optimization_operations_history`	Auto-optimization activity

Implementation

```sql
-- Daily cost by team (using serverless budget policy tags)
SELECT
    usage_date,
    custom_tags:team AS team,
    sku_name,
    SUM(list_cost) AS daily_cost
FROM system.billing.usage
WHERE usage_date >= current_date() - 30
GROUP BY 1, 2, 3
ORDER BY daily_cost DESC;

-- Identify long-running queries
SELECT
    user_name,
    warehouse_id,
    total_duration_ms / 1000 AS duration_seconds,
    statement_text
FROM system.query.history
WHERE total_duration_ms > 300000  -- >5 minutes
ORDER BY total_duration_ms DESC
LIMIT 20;
```

PA-03: Workspace Isolation Strategy

Rule

Separate workspaces by environment (dev/staging/prod) and security boundary. Use Unity Catalog to provide cross-workspace governance.

Why It Matters

- Prevents accidental production changes from development activity
- Enables different security controls per environment
- Supports regulatory requirements for environment segregation
- Allows independent scaling and cost tracking

Workspace Topology

Workspace	Purpose	Access
Production	Production jobs, serving endpoints	Service principals, limited human access
Staging	Pre-production validation, integration testing	Engineering teams, CI/CD
Development	Interactive development, experimentation	All engineers
Sandbox	Exploration, POCs, training	Self-service

When to Create Additional Workspaces

Trigger	Example
Regulatory isolation	HIPAA workloads separate from general analytics
Network boundary	Different VNet/VPC requirements
Cost isolation	Business unit-level billing separation
Team autonomy	Large independent teams needing admin control

When NOT to Split Workspaces

Scenario	Recommendation
Cost attribution only	Use budget policies and tags instead
Schema-level access control	Use UC grants instead
Different compute needs	Use cluster policies instead

PA-04: Lakehouse Federation

> **See also:** DI-02, DI-04, DI-07 for detailed federation guardrails.

Rule

Use Lakehouse Federation to query external databases (PostgreSQL, MySQL, SQL Server, Snowflake, BigQuery, Redshift) directly through Unity Catalog without data replication. Federate for read-only, low-volume access; replicate into Delta Lake for heavy analytics.

Why It Matters

- Eliminates data copying for infrequent or exploratory access patterns
- Provides Unity Catalog governance (lineage, permissions, tags) over external data
- Reduces storage costs for rarely queried datasets
- Enables gradual migration from legacy systems

Implementation

```sql
-- Create a connection to an external database
CREATE CONNECTION postgres_crm TYPE POSTGRESQL
OPTIONS (
    host 'crm-db.company.com',
    port '5432',
    user secret('federation-scope', 'pg-user'),
    password secret('federation-scope', 'pg-password')
);

-- Create a foreign catalog
CREATE FOREIGN CATALOG crm_postgres USING CONNECTION postgres_crm
OPTIONS (database 'crm_production');

-- Query federated data directly
SELECT customer_id, email, created_at
FROM crm_postgres.public.customers
WHERE created_at > '2025-01-01';
```

Federate vs. Replicate Decision

Factor	Federate	Replicate to Delta
Query frequency	Low (<10 queries/day)	High (frequent analytics)
Data volume per query	Small (<1M rows)	Large (full table scans)
Latency tolerance	Higher (remote query)	Low (local Delta)
Governance need	Yes (UC lineage/tags)	Yes (full Delta features)
Write-back needed	No	No (read-only either way)

PA-05: Use Certified Partner Integrations

Rule

When integrating external tools with the Databricks platform, prefer certified partner integrations from the Databricks Partner Connect ecosystem.

Why It Matters

- Certified integrations are tested and validated for compatibility
- Reduces maintenance burden from custom integration code
- Provides supported upgrade paths as platform evolves
- Leverages Unity Catalog and governance features natively

Implementation

Integration Type	Certified Approach	Avoid
BI Tools	Partner Connect (Tableau, Power BI)	Custom JDBC/ODBC drivers
ETL/ELT	Lakeflow Connect, Fivetran, dbt	Custom scripts
Data Quality	Lakeflow Declarative Pipelines expectations	Third-party quality tools outside UC
Orchestration	Lakeflow Jobs, Airflow provider	Custom REST API wrappers

Cost Optimization

COST-01: Budget Policies for Production Workspaces

Rule

All production workspaces must have serverless budget policies configured with spending limits and alerting thresholds.

Why It Matters

- Prevents runaway costs from unbounded serverless usage
- Enables team-level cost accountability
- Provides early warning before budget overruns
- Required for enterprise cost governance

Implementation

1. Navigate to **Settings** > **Compute** > **Serverless budget policies**
2. Create policy per team with spending limits
3. Assign policies to all production service principals and users
4. Configure alert thresholds (e.g., 80% and 100% of budget)

```
-- Monitor budget policy utilization
SELECT
    budget_policy_name,
```

```
    usage_date,
    SUM(list_cost) AS daily_spend
FROM system.billing.usage
WHERE usage_date >= current_date() - 30
GROUP BY 1, 2
ORDER BY 1, 2;
```

COST-02: Quarterly Cost Optimization Reviews

Rule

Conduct quarterly cost optimization reviews using system tables to identify waste, right-sizing opportunities, and optimization actions.

Why It Matters

- System tables provide granular usage and cost data without external tools
- Identifies idle resources, over-provisioned clusters, and inefficient queries
- Tracks optimization impact over time
- Ensures cost governance is continuous, not one-time

Implementation

```
-- Identify underutilized SQL warehouses
SELECT
    warehouse_name,
    COUNT(*) AS query_count,
    AVG(total_duration_ms) / 1000 AS avg_duration_sec,
    SUM(total_duration_ms) / 3600000 AS total_hours
FROM system.query.history
WHERE start_time >= current_date() - 90
GROUP BY warehouse_name
ORDER BY total_hours DESC;

-- Identify expensive jobs for right-sizing
SELECT
    job_id,
    run_name,
    COUNT(*) AS run_count,
    AVG(execution_duration / 1000) AS avg_duration_sec,
    SUM(result_state = 'FAILED') AS failure_count
FROM system.lakeflow.job_run_timeline
WHERE period_start_time >= current_date() - 90
GROUP BY 1, 2
ORDER BY run_count DESC;
```

Review Checklist

Check	Action
Idle warehouses	Auto-stop or consolidate
Over-provisioned clusters	Right-size using metrics
Failed job reruns	Fix root cause, reduce waste
Duplicate pipelines	Consolidate into shared data products
Unused tables	Archive or drop based on lineage

COST-03: Balance Always-On vs Triggered Streaming

Rule

Evaluate whether streaming workloads should run continuously (always-on) or be triggered on a schedule to optimize cost.

Why It Matters

- Always-on streaming incurs continuous compute costs even during low-activity periods
- Triggered (scheduled) streaming can reduce costs by 60-80% for workloads where near-real-time latency is acceptable
- The right choice depends on SLA requirements, data volume patterns, and cost constraints

Decision Table

Latency Requirement	Data Volume	Recommendation
Sub-minute	Continuous high volume	Always-on streaming
Minutes (1-15)	Variable/bursty	Triggered streaming (frequent schedule)
Hourly or longer	Batch-compatible	Triggered streaming or batch

Implementation

```
# Triggered streaming pipeline (cost-optimized)
pipelines:
  bronze_incremental:
    serverless: true
    continuous: false  # Triggered mode
    trigger:
      cron: "0 */15 * * *"  # Every 15 minutes
```

Well-Architected Lakehouse Alignment

This platform aligns with the Databricks Well-Architected Lakehouse Framework across seven pillars:

Pillar	How Addressed	Key Documents
Governance	Unity Catalog, governed tags, data classification, row filters	UC Governance, Data Governance
Security	VNet/VPC injection, Private Link/PSC, CMK, SCIM, Compliance Security Profile	Network Security
Reliability	Time travel, checkpointing, DR procedures	Reliability & DR
Performance	Liquid clustering, predictive optimization, Photon, serverless	UC Tables & Delta Lake, Classic Compute
Cost	Serverless, budget policies, system table monitoring, quarterly reviews	Serverless, Tagging
OpEx	Asset Bundles, CI/CD, monitoring, system tables	Asset Bundles, Monitoring
Interop	Delta Lake open format, UniForm, Delta Sharing, Lakehouse Federation	Delta Sharing, UC Tables & Delta Lake

Related Documents

- Serverless Compute
- Unity Catalog Tables
- Cluster Policies
- Secrets Management
- Architecture Evolution

References

- Azure Databricks Architecture
- Medallion Architecture
- Unity Catalog
- Lakeflow Connect